

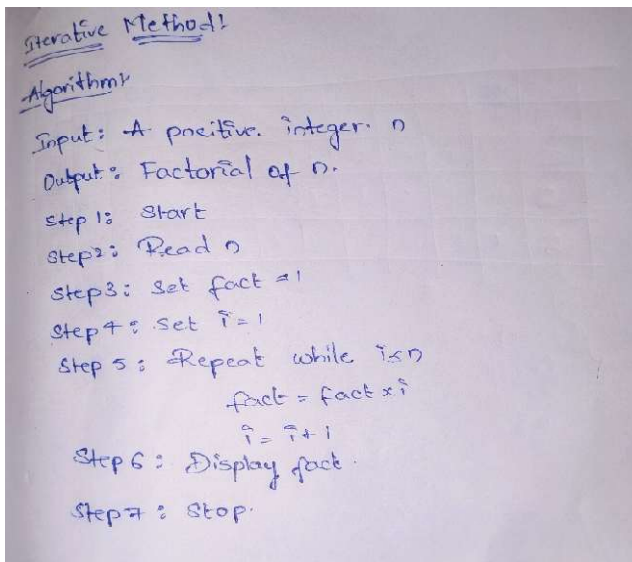


PRACTICAL: 04

Aim: To implement and find the Time analysis of factorial program using iterative and recursive method.

A) Iterative Method

Algorithm:



Program / Code:

```
#include <iostream>  
using namespace std;  
class Factorial {  
public:  
    int fact(int n) {  
        int result = 1;  
        for (int i = 1; i <= n; i++) {  
            result = result * i;  
        }  
        return result;  
    }  
};
```

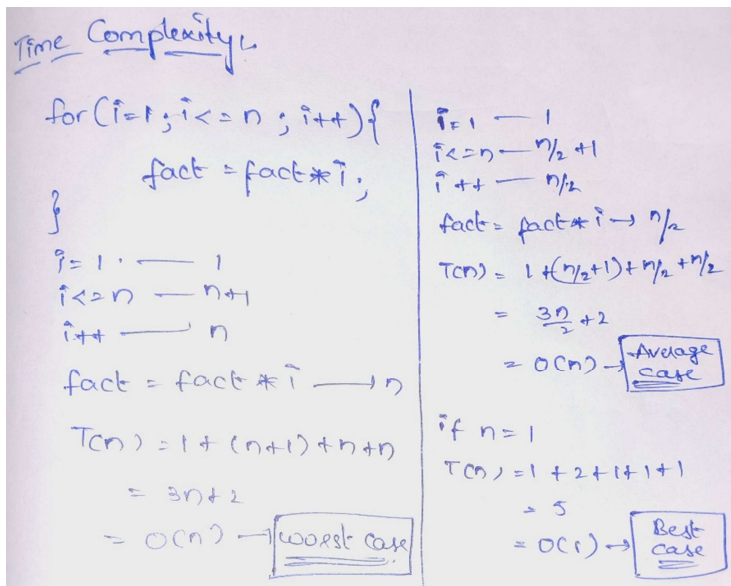
```
int main() {  
    int n, result;  
    Factorial obj;  
    cout << "Enter the value to find its factorial: ";  
    cin >> n;  
    result = obj.fact(n);  
    cout << "Using Iterative Method!" << endl;  
    cout << "Factorial of " << n << " is " << result << endl;  
    return 0;  
}
```

Output:



```
"C:\Users\karth\Downloads\ID x + v  
Enter the value to find its factorial: 8  
Using Iterative Method!  
Factorial of 8 is 40320  
  
Process returned 0 (0x0)   execution time : 7.026 s  
Press any key to continue.
```

Time Complexity:



Time Complexity:

for($i=1$; $i \leq n$; $i++$) {
 fact = fact * i ;
}

$i = 1 \rightarrow 1$
 $i \leq n \rightarrow n+1$
 $i++ \rightarrow n$
fact = fact * $i \rightarrow n$

$T(n) = 1 + (n+1) + n + n$
 $= 3n + 2$
 $= O(n) \rightarrow$ Worst case

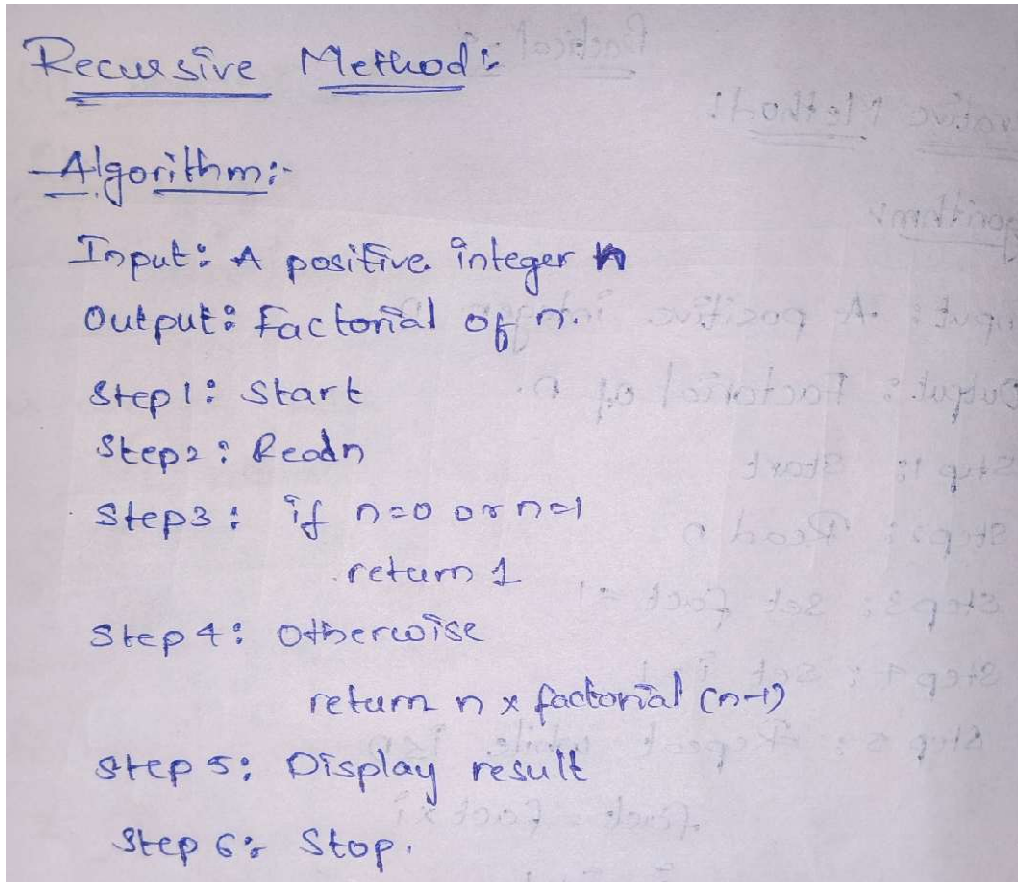
$i = 1 \rightarrow 1$
 $i \leq n \rightarrow n/2 + 1$
 $i++ \rightarrow n/2$
fact = fact * $i \rightarrow n/2$

$T(n) = 1 + (n/2 + 1) + n/2 + n/2$
 $= \frac{3n}{2} + 2$
 $= O(n) \rightarrow$ Average case

if $n = 1$
 $T(n) = 1 + 2 + 1 + 1 + 1$
 $= 5$
 $= O(1) \rightarrow$ Best case

B) Recursive Method:

Algorithm:



Recursive Method:

Algorithm:

Input: A positive integer n

Output: Factorial of n

Step 1: Start

Step 2: Read n

Step 3: If $n = 0$ or $n = 1$
 return 1

Step 4: Otherwise
 return $n \times \text{factorial}(n-1)$

Step 5: Display result

Step 6: Stop

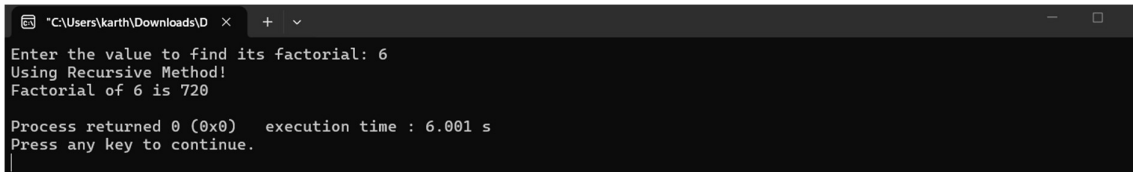
Program / Code:

```
#include <iostream>
using namespace std;
class Factorial {
public:
    int factr(int n) {
        if (n == 0 || n == 1) {
            return 1;
        }
    }
}
```

```
        else {
            int result = n * factr(n - 1);
            return result;
        }
    }
};

int main() {
    int n, result;
    Factorial obj;
    cout << "Enter the value to find its factorial: ";
    cin >> n;
    result = obj.factr(n);
    cout << "Using Recursive Method!" << endl;
    cout << "Factorial of " << n << " is " << result << endl;
    return 0;
}
```

Output:



```
"C:\Users\karth\Downloads\D" x + -
Enter the value to find its factorial: 6
Using Recursive Method!
Factorial of 6 is 720

Process returned 0 (0x0) execution time : 6.001 s
Press any key to continue.
```

Time Complexity:

Time Complexity:-

```

factorial(n)
{
    if (n==0 || n==1) → 1.
        return 1; → 1
    return n * factorial(n-1); → T(n-1)
}

```

$T(n) = T(n-1) + 2$
 $T(n) = T(n-2) + 2 + 2$
 $T(n) = T(n-3) + 2 + 2 + 2$
 $T(n) = 2n$
 $= O(n) \rightarrow$ Worst Case
 when $n=0$ or $n=1$;
 $T(n) = 1$
 $= O(1) \rightarrow$ Best case

$T(n) = T(n-1) + 2$
 $= O(n)$
 $\downarrow$
Average case

Conclusion:

In this practical, we studied and implemented the factorial using both iterative and recursive methods. The iterative method uses a loop, while the recursive method uses function calls. Both methods have $O(n)$ time complexity in the general case. The iterative method uses $O(1)$ space, whereas the recursive method uses $O(n)$ space due to the recursive call stack.